

Rapport de Projet de Fin d'Études

Présenté à :

**L'Institut Supérieur d'Informatique
et de Multimédia de Sfax**

Élaboré par :

**Nouha BOUGHNIM
Wiem BZIOUICH**

**InvisiThreat : Plateforme DevSecOps de détection
automatisée de vulnérabilités par analyses SAST et DAST**

Présenté devant le Jury :

Président : M./Mme *Prénom NOM*

Rapporteur : M./Mme *Prénom NOM*

Examineur : M./Mme *Prénom
NOM*

Encadrant académique :

Mme *Najiba TAGOUGUI*

Encadrant professionnel :

Mr *Houssemeddine HASSAYOUNE*

Organisme d'accueil : **Mobelite**



Année Universitaire : **2025 - 2026**

Dédicace

*À nos parents,
dont le soutien indéfectible, les sacrifices silencieux
et la confiance sans faille ont rendu possible
chaque étape de ce parcours.*

*À nos enseignants de l'ISIMS,
qui nous ont transmis bien plus que des connaissances :
une rigueur, une méthode et le goût de l'excellence.*

*À nos encadrantes et encadrants,
Mme Najiba TAGOUGUI et M. Houssemeddine HASSAYOUNE,
pour leur disponibilité, leurs orientations éclairées
et la confiance accordée tout au long de ce projet.*

*À nos collègues et amis,
compagnons de longues nuits de travail,
dont l'encouragement mutuel a transformé
chaque difficulté en opportunité d'apprentissage.*

*À toute personne qui croit que la sécurité informatique
n'est pas une contrainte, mais une responsabilité.*

Nouha & Wiem

Remerciements

La réalisation de ce projet de fin d'études n'aurait pas été possible sans le concours bienveillant de nombreuses personnes à qui nous tenons à exprimer notre sincère gratitude.

Nous adressons en premier lieu nos remerciements les plus chaleureux à notre encadrante académique, **Mme Najiba TAGOUGUI**, pour la qualité de son encadrement, la pertinence de ses conseils scientifiques et sa disponibilité constante tout au long de ce travail. Sa rigueur intellectuelle et son soutien méthodologique ont été déterminants dans la conduite et la réussite de ce projet.

Nous remercions également notre encadrant professionnel, **M. Houssemeddine HASSAYOUNE**, ingénieur chez Mobelite, pour nous avoir accueillis au sein de son équipe, partagé son expertise terrain en matière de sécurité applicative et orienté nos choix techniques avec une vision résolument pragmatique. Sa connaissance des enjeux DevSecOps a été une source d'inspiration continue.

Nos remerciements vont aussi à la direction et aux équipes de **Mobelite** pour avoir mis à notre disposition un environnement de travail stimulant, un accès aux ressources nécessaires et un cadre professionnel propice à l'innovation.

Nous tenons à remercier les membres du jury, qui ont accepté d'évaluer ce travail et dont les remarques contribueront à son enrichissement. Leur expertise représente pour nous une reconnaissance précieuse de l'effort fourni.

Nous exprimons notre profonde gratitude à l'ensemble du corps enseignant de l'**Institut Supérieur d'Informatique et de Multimédia de Sfax**, dont la formation solide nous a dotés des compétences techniques et analytiques nécessaires pour mener à bien un projet de cette envergure.

Enfin, nous remercions nos familles respectives pour leur soutien moral indéfectible, leur patience et leur encouragement tout au long de ces années d'études. Ce travail leur appartient autant qu'à nous.

Liste des Abréviations

API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
DAST	Dynamic Application Security Testing
JWT	JSON Web Token
OWASP	Open Worldwide Application Security Project
RBAC	Role-Based Access Control
SAST	Static Application Security Testing
SCA	Software Composition Analysis
SQL	Structured Query Language
UI	User Interface
UX	User Experience
ZAP	Zed Attack Proxy

Table des matières

Dédicace	i
Remerciements	ii
Liste des Abréviations	iii
Introduction Générale	1
1 Cadre général du projet	2
1.1 Introduction	2
1.2 Présentation de l'organisme d'accueil	2
1.3 Présentation du projet InvisiThreat	3
1.3.1 Contexte général	3
1.3.2 Définition de la mission	3
1.3.3 Problématique	3
1.3.4 Objectifs du projet	4
1.3.5 Périmètre du projet	4
1.4 Étude et critique de l'existant	5
1.4.1 Panorama des solutions existantes	5
1.4.1.1 SonarQube	5
1.4.1.2 Burp Suite Professional	5
1.4.1.3 Checkmarx	5
1.4.1.4 Snyk	6
1.4.1.5 OWASP ZAP	6
1.4.2 Critiques et recommandations	6
1.4.2.1 Synthèse comparative	6
1.4.2.2 Critique de l'existant	7
1.4.3 Solution proposée	7
1.5 Méthodologie de développement	8
1.5.1 Choix méthodologique : Scrum	8

1.5.1.1	Étude comparative des méthodologies	8
1.5.1.2	Valeurs fondamentales de l'Agile	8
1.5.1.3	Présentation de Scrum	8
1.5.1.4	Les trois piliers de Scrum	9
1.5.2	Rôles et cérémonies	9
1.5.2.1	Artefacts de Scrum	10
1.6	Conclusion	10
2	Sprint 0 – Initialisation et étude technique	11
2.1	Introduction	11
2.2	Identification des acteurs	11
2.3	Analyse des besoins	12
2.3.1	Besoins fonctionnels	12
2.3.2	Besoins non fonctionnels	14
2.3.3	Diagramme global des cas d'utilisation	14
2.4	Gestion du projet avec Scrum	15
2.4.1	Product Backlog	15
2.4.1.1	Structuration en epics	15
2.4.1.2	Technique de priorisation du Product Backlog	15
2.4.1.3	Synthèse du Product Backlog	16
2.4.2	Planification des sprints	20
2.5	Étude technique	21
2.5.1	Environnement matériel	21
2.5.2	Environnement logiciel	21
2.5.3	Technologies et outils utilisés	22
2.5.4	Architecture du système	24
2.5.4.1	Architecture logicielle	24
2.5.4.2	Architecture physique	24
2.6	Diagramme de déploiement	25
2.7	Conclusion	25
3	Sprint 1 – Sécurité et gouvernance	27
3.1	Introduction	27
3.2	Objectifs du sprint et planification des tâches	27
3.2.1	Objectifs du sprint	27
3.2.2	Sprint Backlog	27

3.3	Technologies et concepts utilisés	27
3.3.1	JWT et OAuth2 (FastAPI Security)	27
3.3.2	TOTP et RBAC	28
3.4	Analyse et spécification des besoins	28
3.4.1	Diagramme de cas d'utilisation du sprint	28
3.4.2	Description détaillée des cas d'utilisation	28
3.5	Modélisation conceptuelle	29
3.5.1	Diagramme de classes	29
3.5.2	Diagrammes de séquence	29
3.6	Réalisation	29
3.6.1	Authentification JWT	29
3.6.2	RBAC et gestion des rôles	29
3.6.3	Audit logs et 2FA	30
3.7	Test et validation	30
3.7.1	Tests réalisés	30
3.7.2	Validation des fonctionnalités	30
3.7.3	Gestion des versions / commits	30
3.8	Conclusion	30
3.8.1	Diagramme IA/LLM	30
3.8.2	Diagramme d'activités	31
3.9	Configuration du projet	31
3.10	Bonnes pratiques	31
3.11	Conclusion	32
4	Sprint 2 – Projets et intégrations	33
4.1	Introduction	33
4.2	Objectifs du sprint et planification des tâches	33
4.2.1	Objectifs du sprint	33
4.2.2	Sprint Backlog	33
4.3	Technologies et concepts utilisés	33
4.4	Analyse détaillée des besoins	34
4.4.1	Diagramme de cas d'utilisation du sprint	34
4.4.2	Description détaillée des cas d'utilisation	34
4.5	Modélisation conceptuelle	34
4.5.1	Diagramme de classes	34
4.5.2	Diagrammes de séquence	35

4.6	Réalisation	35
4.6.1	Gestion des projets et des dépôts	35
4.6.2	Intégrations OAuth et clés API	35
4.7	Test et validation	35
4.7.1	Tests réalisés	35
4.7.2	Validation	35
4.8	Conclusion	36
5	Sprint 3 – SAST et SCA	37
5.1	Introduction	37
5.2	Objectifs du sprint et planification des tâches	37
5.2.1	Objectifs du sprint	37
5.2.2	Sprint Backlog	37
5.3	Analyse détaillée des besoins	38
5.3.1	Diagramme de cas d'utilisation du sprint	38
5.3.2	Description textuelle des cas d'utilisation	38
5.4	Modélisation conceptuelle	38
5.4.1	Diagramme de classes	38
5.4.2	Diagrammes de séquence	39
5.5	Réalisation	39
5.5.1	Intégration SAST (Semgrep, Bandit)	39
5.5.2	Intégration SCA (OSV)	39
5.6	Test et validation	39
5.6.1	Tests réalisés	39
5.6.2	Validation	39
5.7	Conclusion	40
6	Sprint 4 – DAST, orchestration et IA	41
6.1	Introduction	41
6.2	Objectifs du sprint et planification des tâches	41
6.2.1	Objectifs du sprint	41
6.2.2	Sprint Backlog	41
6.3	Analyse détaillée des besoins	42
6.3.1	Diagramme de cas d'utilisation du sprint	42
6.3.2	Description textuelle des cas d'utilisation	42
6.4	Modélisation conceptuelle	42

6.4.1	Diagramme de classes	42
6.4.2	Diagrammes de séquence	43
6.5	Réalisation	43
6.5.1	Intégration DAST (OWASP ZAP)	43
6.5.2	Orchestration et workflow	43
6.6	Test et validation	43
6.6.1	Tests réalisés	43
6.6.2	Validation	43
6.7	Conclusion	44
CONCLUSION GÉNÉRALE		45
BIBLIOGRAPHIE		46

0

Table des figures

1.1	Logo de l'entreprise Mobelite.	2
1.2	Logo SonarQube	5
1.3	Logo Burp Suite	5
1.4	Logo Checkmarx	5
1.5	Logo Snyk	6
1.6	Logo OWASP ZAP	6
1.7	Cycle Scrum appliqué au projet InvisiThreat.	9
2.1	Diagramme de cas d'utilisation global d'InvisiThreat.	15
2.2	Architecture logicielle en couches d'InvisiThreat.	24
2.3	Architecture physique – déploiement Docker Compose d'InvisiThreat.	25
2.4	Diagramme de déploiement Docker Compose d'InvisiThreat.	25
3.1	Diagramme de cas d'utilisation – Sprint 1 (placeholder).	28
3.2	Diagramme de classes – Sprint 1 (placeholder).	29
3.3	Diagrammes de séquence – Sprint 1 (placeholder).	29
3.4	Diagramme IA/LLM (placeholder).	31
3.5	Diagramme d'activités (placeholder).	31
4.1	Diagramme de cas d'utilisation – Sprint 2 (placeholder).	34
4.2	Diagramme de classes – Sprint 2 (placeholder).	34
4.3	Diagrammes de séquence – Sprint 2 (placeholder).	35
5.1	Diagramme de cas d'utilisation – Sprint 3 (placeholder).	38
5.2	Diagramme de classes – Sprint 3 (placeholder).	38
5.3	Diagrammes de séquence – Sprint 3 (placeholder).	39
6.1	Diagramme de cas d'utilisation – Sprint 4 (placeholder).	42
6.2	Diagramme de classes – Sprint 4 (placeholder).	42
6.3	Diagrammes de séquence – Sprint 4 (placeholder).	43

0

Liste des tableaux

1.1	Fiche d'identité de Mobelite	3
1.2	Comparatif des solutions de sécurité applicative	6
1.3	Comparaison des méthodologies de développement	8
2.1	Acteurs du système InvisiThreat et leurs responsabilités	12
2.2	Product Backlog – Gouvernance et gestion des accès	17
2.3	Product Backlog – Scans et orchestration	18
2.4	Product Backlog – Workflow, IA, reporting et CI/CD	19
2.5	Planification et livrables des sprints InvisiThreat	20
2.6	Configuration matérielle des postes de travail	21
2.7	Environnement logiciel utilisé pour le développement	22
2.8	Stack technique d'InvisiThreat – technologies et justifications	23
3.1	Sprint Backlog – Sprint 1	27
4.1	Sprint Backlog – Sprint 2	33
5.1	Sprint Backlog – Sprint 3	37
6.1	Sprint Backlog – Sprint 4	41

0

Introduction Générale

À l'ère de la transformation numérique, la sécurité des systèmes d'information est devenue un enjeu stratégique majeur pour les organisations de toute taille. L'explosion du nombre d'applications web, conjuguée à la sophistication croissante des cyberattaques, expose quotidiennement les entreprises à des risques considérables : vol de données sensibles, interruption de services, atteinte à la réputation et pertes financières. Selon les rapports récents des organismes spécialisés, le nombre de vulnérabilités découvertes chaque année ne cesse de croître, tandis que les délais d'exploitation par des acteurs malveillants se réduisent drastiquement.

Face à cette réalité, les approches traditionnelles de sécurité — reposant sur des audits ponctuels et des interventions manuelles — montrent leurs limites. Elles peinent à s'adapter au rythme imposé par les cycles de développement modernes, où le code évolue en continu et les mises en production se succèdent à cadence soutenue. Il en résulte une fenêtre de vulnérabilité persistante, durant laquelle les failles applicatives demeurent exposées sans être détectées ni corrigées.

C'est dans ce contexte qu'émerge la nécessité d'intégrer la sécurité au cœur même du cycle de développement logiciel, selon le paradigme **DevSecOps**. Cette approche vise à automatiser la détection des vulnérabilités dès les premières étapes du développement, rendant ainsi la sécurité continue, proactive et mesurable. Cependant, malgré l'existence d'outils d'analyse isolés, les équipes techniques se heurtent souvent à l'absence d'une plateforme unifiée capable d'orchestrer ces analyses, d'en consolider les résultats et d'en restituer une vision claire et priorisée.

C'est précisément pour répondre à ce besoin, identifié au sein de l'entreprise **Mobelite**, qu'a été initié le projet **InvisiThreat**. La solution est une plateforme DevSecOps intégrée combinant **SAST**, **DAST** et **SCA** dans un pipeline unifié. Elle orchestre les scans de manière asynchrone (*Celery/Redis*), normalise les résultats, applique un **scoring de risque**, gère un **workflow de vulnérabilités** et diffuse des **notifications temps réel**. L'intégration **GitHub** (OAuth) et l'usage de **clés API** rendent possible le déclenchement des analyses depuis un CLI ou un pipeline CI/CD.

L'assistance intelligente constitue un élément central : un module IA local (*Ollama*, modèles *Mistral/Qwen*) est exposé dans l'interface via l'action *Assist with AI*. Il explique les vulnérabilités, simplifie les messages techniques, propose des recommandations et guide le développeur vers la correction.

Le développement de ce projet a été conduit selon une méthodologie **Scrum**, favorisant une progression itérative, une collaboration continue avec les parties prenantes et une capacité d'adaptation aux besoins évolutifs du terrain.

Chapitre

1

Cadre général du projet

1.1 Introduction

La sécurité applicative s'impose aujourd'hui comme une préoccupation centrale pour toute organisation reposant sur des systèmes informatiques. L'essor des architectures orientées services et l'adoption des pratiques DevOps ont profondément reconfiguré le paysage des menaces numériques : la sécurité doit désormais être intégrée de manière continue et automatisée à chaque stade du pipeline de livraison.

Ce chapitre pose le cadre général du projet **InvisiThreat** : présentation de l'organisme d'accueil, problématique de sécurité, étude critique de l'existant, solution proposée et méthodologie de développement retenue.

1.2 Présentation de l'organisme d'accueil

Mobelite est une entreprise spécialisée dans la conception et le développement d'applications web et mobiles à forte valeur ajoutée. Elle accompagne ses clients dans la transformation numérique en proposant des solutions sur mesure fondées sur des architectures orientées services.



FIGURE 1.1 – Logo de l'entreprise Mobelite.

Son environnement technique repose sur Git, Docker, des pipelines CI/CD et des API REST consommées par des frontends React ou des clients mobiles — une stack efficace qui génère néanmoins une surface d'attaque étendue. Avant InvisiThreat, les audits de sécurité étaient conduits de manière ponctuelle et hétérogène, sans outillage unifié ni traçabilité structurée, ce qui rendait difficile la mesure de l'évolution du risque. Ce constat a motivé la création d'un outil interne capable de **standardiser, automatiser et centraliser** la détection des failles à l'échelle du portefeuille applicatif.

TABLE 1.1 – Fiche d'identité de Mobelite

Nom de l'entreprise	Mobelite
Secteur d'activité	Développement d'applications web et mobiles
Date de création	[à compléter]
Adresse	[à compléter]
Site web	[à compléter]
Email	[à compléter]

1.3 Présentation du projet InvisiThreat

1.3.1 Contexte général

L'accélération des cycles de livraison logicielle impose une approche radicalement différente des audits périodiques traditionnels. C'est dans ce contexte qu'a émergé le paradigme **DevSecOps**, qui intègre la sécurité dès la conception et à chaque étape du pipeline. Deux familles d'analyses en constituent le socle :

- **SAST** (*Static Application Security Testing*) : inspection du code source pour identifier des patterns dangereux, sans exécuter l'application.
- **DAST** (*Dynamic Application Security Testing*) : test de l'application en cours d'exécution pour détecter les failles d'exposition réelles.
- **SCA** (*Software Composition Analysis*) : analyse des dépendances pour identifier les vulnérabilités connues dans les bibliothèques tierces.

À ces trois dimensions s'ajoute un **assistant IA** basé sur des modèles LLM locaux (Ollama, Mistral/Qwen) chargé de contextualiser les alertes, de formuler des recommandations exploitables et d'accompagner la remédiation via l'action *Assist with AI*.

1.3.2 Définition de la mission

Dans le cadre de notre projet de fin d'études pour l'obtention du diplôme Ingénieur à l'Institut supérieur d'informatique et de multimédia de Sfax (ISIMS), Mobelite nous a confié la conception et le développement de la plateforme **InvisiThreat**. Cette plateforme vise à centraliser et automatiser les analyses de sécurité applicative (SAST, DAST, SCA), à enrichir les résultats par intelligence artificielle et à s'intégrer nativement dans les pipelines DevOps de l'entreprise.

1.3.3 Problématique

Les pratiques de sécurité traditionnelles reposent sur des audits manuels périodiques, structurellement inadaptés aux cadences des pipelines CI/CD modernes. Chaque déploiement peut introduire de nouvelles vulnérabilités sans que les mécanismes de contrôle classiques ne les interceptent à temps.

La multiplication des outils de scan isolés aggrave la situation : formats hétérogènes (SARIF, JSON propriétaire, HTML), nomenclatures de sévérité incompatibles, corrélation manuelle laborieuse. L'absence d'historique centralisé par projet empêche par ailleurs toute mesure objective de la maturité sécurité dans le temps.

La problématique centrale est donc : **comment orchestrer de manière cohérente et automatisée des analyses SAST et DAST, enrichir leurs résultats par intelligence artificielle, normaliser les alertes selon une nomenclature unifiée et fournir aux équipes une vision priorisée, contextualisée et historisée du risque applicatif, intégrable dans un pipeline DevOps existant ?**

1.3.4 Objectifs du projet

InvisiThreat poursuit des objectifs complémentaires :

1. **Centraliser** l'orchestration des analyses SAST, DAST et SCA via un point d'entrée unique.
2. **Normaliser** les résultats vers un modèle interne commun et calculer un **scoring de risque** exploitable.
3. **Enrichir** les vulnérabilités via un assistant IA local fournissant des explications et recommandations contextualisées.
4. **Gérer un workflow de vulnérabilités** (états, assignation, commentaires) et assurer la **traçabilité** par audit logs.
5. **Intégrer** GitHub (OAuth) et des **clés API** pour déclencher les scans depuis un CLI ou un pipeline CI/CD.
6. **Notifier** en temps réel et fournir des exports exploitables (PDF/JSON) pour l'audit.
7. **Historiser** les scans par projet et par version afin de suivre l'évolution du risque.

1.3.5 Périmètre du projet

InvisiThreat couvre les dimensions suivantes :

- **SAST** : moteurs **Semgrep** (analyse polyglotte par règles) et **Bandit** (inspection Python).
- **SCA** : intégration de **OSV** pour l'analyse des dépendances.
- **DAST** : orchestration via l'**API REST d'OWASP ZAP** en mode daemon.
- **Assistant IA** : modèles LLM locaux via **Ollama** avec action *Assist with AI*.
- **Gouvernance** : JWT, RBAC, 2FA TOTP, audit logs et workflow de vulnérabilités.
- **Intégrations** : GitHub OAuth, clés API et CLI scanner.
- **Stack technique** : backend FastAPI, workers Celery, Redis, PostgreSQL, frontend React, notifications temps réel via Socket.IO, conteneurisation Docker Compose.

Le projet se concentre exclusivement sur la sécurité applicative web et ne couvre pas les infrastructures réseau ni les composants matériels.

1.4 Étude et critique de l'existant

1.4.1 Panorama des solutions existantes

Une analyse des principaux outils du marché a été conduite selon trois catégories : SAST, DAST et plateformes intégrées.

1.4.1.1 SonarQube

Plateforme d'inspection continue de la qualité et de la sécurité du code, largement adoptée en CI/CD. La version Community offre une couverture des règles de sécurité réduite, sans capacités DAST ni enrichissement intelligent. Les éditions Enterprise restent coûteuses pour les structures intermédiaires.



FIGURE 1.2 – Logo SonarQube

1.4.1.2 Burp Suite Professional

Référence de l'industrie en analyse dynamique web. Son moteur de scan actif est reconnu pour sa précision. Cependant, les licences annuelles sont élevées, l'automatisation complète est réservée à l'édition Pro et l'outil ne dispose d'aucune capacité SAST ni d'historique de scans exploitable.



FIGURE 1.3 – Logo Burp Suite

1.4.1.3 Checkmarx

Solution SAST entreprise offrant une analyse approfondie du code source multi-langages. Son coût prohibitif, son fonctionnement en boîte noire et l'absence native de DAST et d'intelligence artificielle limitent son adéquation au contexte du projet.



FIGURE 1.4 – Logo Checkmarx

1.4.1.4 Snyk

Plateforme SaaS combinant SCA et SAST avec une forte orientation vers la sécurité des dépendances. Sa nature cloud impose une transmission du code vers des serveurs externes (contrainte de confidentialité), et elle ne propose ni DAST ni enrichissement par IA.



FIGURE 1.5 – Logo Snyk

1.4.1.5 OWASP ZAP

Outil open source de référence pour l'analyse dynamique. Puissant et extensible via API REST, il ne couvre néanmoins pas l'analyse statique et ne propose aucun historique centralisé ni enrichissement intelligent des alertes.



FIGURE 1.6 – Logo OWASP ZAP

1.4.2 Critiques et recommandations

1.4.2.1 Synthèse comparative

Le tableau 1.2 présente une comparaison des solutions selon les critères clés du projet.

TABLE 1.2 – Comparatif des solutions de sécurité applicative

Outil	SAST	DAST	Open Source	Enrichissement IA	Historique
SonarQube	✓	✗	Partiel	✗	Partiel
Burp Suite	✗	✓	✗	✗	Limité
Checkmarx	✓	✗	✗	✗	✓
Snyk	Partiel	✗	Partiel	✗	✓
OWASP ZAP	✗	✓	✓	✗	✗
InvisiThreat	✓	✓	✓	✓	✓

1.4.2.2 Critique de l'existant

L'analyse met en évidence une lacune commune à l'ensemble de l'écosystème : aucune solution ne couvre simultanément l'analyse statique, l'analyse dynamique, la restitution unifiée des résultats et leur enrichissement par intelligence artificielle, au sein d'une interface accessible sans infrastructure dédiée coûteuse.

a) Avantages des solutions existantes

- Outils matures avec une large base d'utilisateurs et une documentation étendue.
- Certaines solutions (OWASP ZAP, Semgrep) sont open source et gratuitement accessibles.
- Intégrations CI/CD disponibles pour les principales plateformes (GitHub, GitLab, Jenkins).

b) Limites des solutions existantes

- Aucune solution ne couvre simultanément SAST, DAST et SCA dans une interface unifiée.
- Absence d'enrichissement par intelligence artificielle pour la contextualisation des alertes.
- Formats de résultats hétérogènes (SARIF, JSON, HTML) rendant la corrélation manuelle.
- Coût prohibitif des solutions entreprise ou envoi du code vers des serveurs externes (SaaS).
- Absence d'historique centralisé empêchant le suivi de l'évolution du risque dans le temps.

1.4.3 Solution proposée

Face à ces lacunes, **InvisiThreat** propose une approche intégrée reposant sur cinq principes fondateurs :

- **Pipeline unifié** : un point d'entrée unique pour déclencher les scans SAST (Semgrep, Bandit), SCA (OSV) et DAST (OWASP ZAP).
- **Normalisation et scoring** : alertes converties vers un modèle commun, sévérité unifiée et score de risque par projet.
- **Enrichissement IA** : contextualisation des alertes et recommandations de remédiation via un assistant LLM local.
- **Workflow et traçabilité** : suivi des vulnérabilités par état, assignation, historique et audit logs.
- **Accessibilité opérationnelle** : notifications temps réel, exports PDF/JSON, GitHub OAuth, clés API et API REST pour l'intégration CI/CD.

1.5 Méthodologie de développement

1.5.1 Choix méthodologique : Scrum

La nature d’InvisiThreat — intégration de moteurs tiers, ajustement itératif des règles de détection, incorporation d’un LLM — impose une approche capable d’absorber les incertitudes techniques et d’intégrer progressivement les retours utilisateurs. Une méthodologie **Agile Scrum** s’est naturellement imposée.

Les approches agiles considèrent qu’une planification exhaustive en amont est peu adaptée aux projets complexes : spécifier l’intégralité d’un produit dès le départ limite la capacité d’adaptation. Elles privilégient donc des cycles courts et répétés, en découpant le projet en mini-projets priorisés selon la valeur et le besoin, afin d’obtenir des retours rapides et d’ajuster la trajectoire en continu.

1.5.1.1 Étude comparative des méthodologies

Le tableau 1.3 compare les principales méthodologies envisagées pour le projet.

TABLE 1.3 – Comparaison des méthodologies de développement

Critère	Cycle en V / RUP Cascade		Scrum
Flexibilité	Faible	Moyenne	Forte
Adaptation au changement	Difficile	Possible	Naturelle
Livraisons	Unique en fin	Phases	Itératives (sprints)
Communication	Formelle	Structurée	Continue
Adapté à ce projet	Non	Partiellement	Oui

1.5.1.2 Valeurs fondamentales de l’Agile

Les méthodes agiles s’appuient sur quatre valeurs fondamentales :

- l’interaction entre acteurs plutôt que les processus et les outils ;
- un produit opérationnel plutôt qu’une documentation pléthorique ;
- la collaboration avec le client plutôt que la négociation contractuelle ;
- la réactivité au changement plutôt que le suivi rigide d’un plan.

1.5.1.3 Présentation de Scrum

Scrum est une méthode agile apparue au début des années 2000, dont le nom est emprunté au rugby et signifie « mêlée ». Elle repose sur un découpage du projet en itérations courtes appelées *sprints*, avec un objectif clair à chaque cycle. Le principe est de livrer en priorité les fonctionnalités à forte valeur, tout en garantissant la qualité et un retour utilisateur rapide.

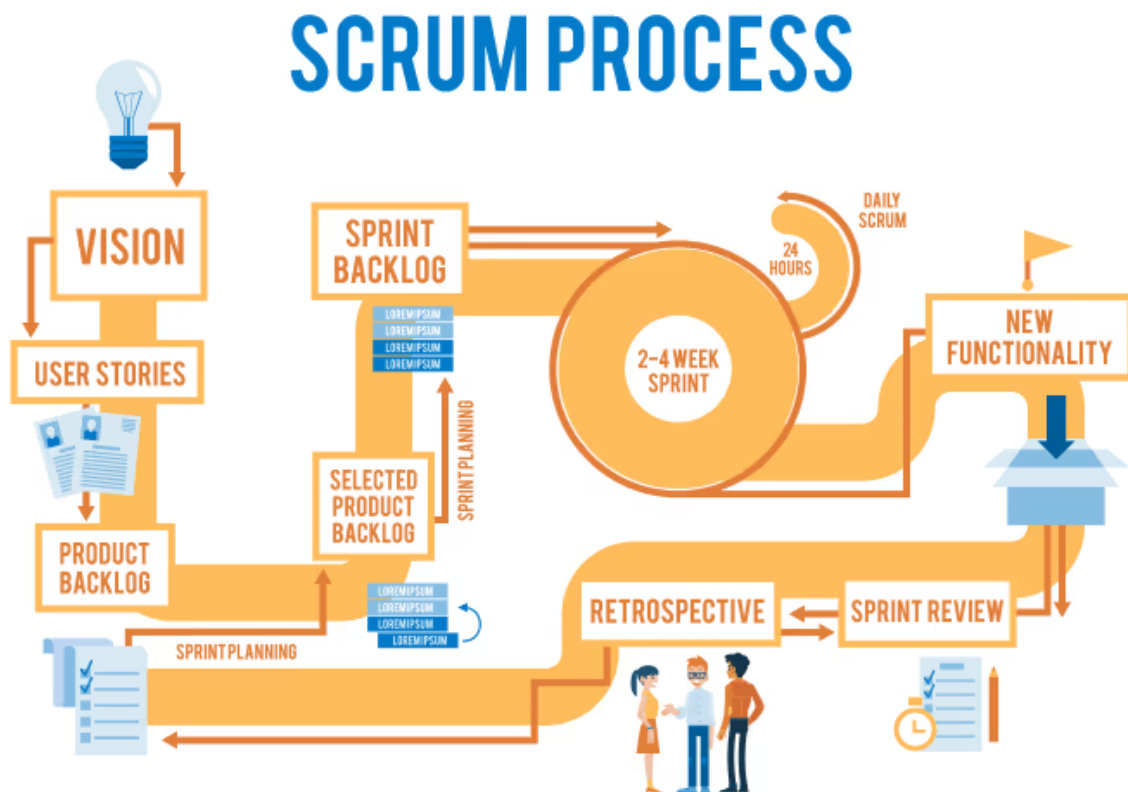


FIGURE 1.7 – Cycle Scrum appliqué au projet InvisiThreat.

1.5.1.4 Les trois piliers de Scrum

Scrum repose sur trois piliers fondamentaux :

- **Transparence** : les aspects importants du processus doivent être visibles et compris par tous les membres de l'équipe.
- **Inspection** : un bilan régulier est réalisé sur les résultats produits afin de détecter tout écart indésirable.
- **Adaptation** : lorsqu'un écart est constaté, le processus est ajusté par des actions correctives ciblées.

1.5.2 Rôles et cérémonies

Scrum définit trois rôles, associés au projet comme suit :

- **Product Owner** (priorisation du backlog selon la valeur et le risque technique) : *****.
- **Scrum Master** (garant du cadre et facilitateur) : *****.
- **Équipe de développement** (auto-organisée, responsable de la qualité des livrables) : Wiem Bziouich, Nouha Boughnim.

Les cérémonies structurantes sont le *Sprint Planning*, le *Daily Scrum*, la *Sprint Review* et la *Sprint Retrospective*, assurant une visibilité continue sur l'avancement et les obstacles.

1.5.2.1 Artefacts de Scrum

- **Product Backlog** : liste priorisée de toutes les fonctionnalités (user stories) du produit, sous la responsabilité du Product Owner.
- **Sprint Backlog** : sous-ensemble du Product Backlog sélectionné pour un sprint donné, avec la liste des tâches associées.
- **Burndown Chart** : courbe graphique permettant de suivre la progression de l'équipe par rapport à l'objectif du sprint.
- **TaskBoard** : tableau (To Do / In Progress / Done) visible par tous les acteurs, reflétant l'état des tâches en temps réel.

1.6 Conclusion

Ce chapitre a posé le cadre général du projet InvisiThreat : identification des lacunes des pratiques de sécurité existantes chez Mobelite, analyse critique des solutions du marché et présentation de la plateforme proposée combinant SAST, DAST et enrichissement par LLM au sein d'un environnement unifié, traçable et intégrable dans les workflows DevOps. La méthodologie Scrum retenue garantit une progression maîtrisée et une validation régulière des modules critiques.

Le chapitre suivant est consacré à l'analyse et à la conception détaillée du système : spécification des besoins fonctionnels et non fonctionnels, identification des acteurs et modélisation UML.

Chapitre

2

Sprint 0 – Initialisation et étude technique

2.1 Introduction

Dans la continuité du chapitre précédent, le **Sprint 0** constitue la phase de fondation du projet InvisiThreat. Il a pour objectif de formaliser les bases nécessaires à un développement maîtrisé : identification des acteurs, spécification des besoins fonctionnels et non fonctionnels, structuration du Product Backlog, planification des sprints et étude technique initiale portant sur les environnements, les technologies retenues et l'architecture du système.

2.2 Identification des acteurs

Avant d'entamer la présentation des diagrammes, il convient d'identifier les acteurs en interaction avec le système. En modélisation UML, un acteur désigne un rôle joué par une entité externe — utilisateur humain, dispositif matériel ou système tiers — qui communique directement avec le système étudié. Il peut consulter ou modifier l'état du système en échangeant des messages porteurs de données.

L'identification préalable des acteurs permet de structurer les responsabilités, de délimiter les périmètres d'accès et de justifier les mécanismes d'autorisation et de traçabilité qui seront mis en place. InvisiThreat implique cinq acteurs distincts, dont les rôles sont synthétisés dans le tableau 2.1.

TABLE 2.1 – Acteurs du système InvisiThreat et leurs responsabilités

Acteur	Rôle principal
Admin	Gouvernance globale de la plateforme, gestion des comptes utilisateurs, configuration des intégrations CI/CD et supervision des politiques de sécurité.
Développeur	Déclenchement des scans sur ses projets rattachés, consultation des résultats et application des correctifs identifiés.
Security Manager	Pilotage de la posture sécurité, priorisation des alertes selon le risque, production et diffusion de rapports de sécurité.
Viewer	Accès en lecture seule aux tableaux de bord et aux rapports, sans pouvoir intervenir sur les configurations ou les états des vulnérabilités.
Systèmes externes	Pipelines CI/CD, dépôts Git et moteur OWASP ZAP interagissant avec la plateforme via l'API REST pour déclencher des scans ou récupérer des résultats.

2.3 Analyse des besoins

L'analyse des besoins d'InvisiThreat se situe à l'intersection de contraintes métier, techniques et sécuritaires. Sur le plan métier, l'enjeu central est de garantir une continuité de livraison logicielle sans compromettre la sécurité, tout en offrant aux équipes une visibilité claire et exploitable sur le niveau de risque par projet. Sur le plan technique, la solution doit s'intégrer à l'écosystème DevOps existant (pipelines CI/CD, dépôts Git, environnements conteneurisés) et traiter des volumes d'alertes croissants sans dégrader les temps de réponse.

Les besoins des équipes sécurité et développement convergent vers une même exigence : disposer d'un processus structuré de gestion des vulnérabilités, fondé sur la traçabilité, la priorisation et l'automatisation. Cela suppose un modèle de données commun pour les alertes, une historisation par version applicative, et des mécanismes d'orchestration découplant les analyses lourdes en tâches asynchrones. L'efficacité de la plateforme dépend également de sa capacité à réduire le bruit (faux positifs) et à transformer les résultats bruts en décisions de remédiation pilotables — d'où l'importance d'une couche d'enrichissement par intelligence artificielle.

2.3.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les capacités que la plateforme doit offrir pour couvrir l'ensemble du cycle de gestion des vulnérabilités. Chaque fonctionnalité est motivée par un objectif métier précis, un mécanisme technique sous-jacent et un apport explicite en termes de sécurité ou de gouvernance.

- **Authentification JWT** : authentification stateless reposant sur des jetons d'accès à courte durée de vie et des mécanismes de rafraîchissement, afin d'équilibrer sécurité et performance. La contrainte principale porte sur la gestion des sessions révoquées et la synchronisation temporelle des services.

- **Authentification forte (2FA TOTP)** : second facteur d'authentification basé sur des codes temporels pour les comptes à privilèges, réduisant l'exposition aux compromissions par vol d'identifiants.
- **Contrôle d'accès par rôles (RBAC)** : séparation des responsabilités via une matrice de droits évaluée à chaque requête, limitant l'accès excessif et facilitant l'audit des permissions.
- **Journalisation des actions sensibles (Audit logs)** : enregistrement horodaté des opérations critiques — connexion, lancement de scan, changement d'état de vulnérabilité — afin de satisfaire aux exigences de conformité et d'investigation.
- **Gestion des projets et des dépôts** : centralisation des projets, dépôts Git et URLs cibles, avec paramètres de scan configurables et stockage sécurisé des secrets (tokens, clés).
- **Intégration GitHub OAuth** : rattachement des dépôts et branches aux projets pour automatiser la collecte de métadonnées et simplifier l'authentification des équipes de développement.
- **Clés API** : authentification technique des appels provenant de pipelines CI/CD ou d'un CLI, sans exposition d'identifiants utilisateur.
- **Orchestration asynchrone des scans** : planification, exécution en file d'attente, suivi de progression et arrêt des analyses, avec gestion de la concurrence pour garantir la stabilité opérationnelle en charge élevée.
- **Analyse statique SAST** : détection de patterns de code vulnérables via Semgrep et Bandit, avec normalisation des sorties hétérogènes vers un modèle commun. La maîtrise des faux positifs constitue l'enjeu principal.
- **Analyse des dépendances (SCA)** : identification des composants tiers vulnérables par interrogation de la base OSV, alignée sur les identifiants CVE.
- **Analyse dynamique DAST** : pilotage du moteur OWASP ZAP par API REST, avec gestion du contexte applicatif et de l'authentification. La durée d'exécution et la disponibilité des environnements cibles constituent les contraintes majeures.
- **Normalisation des vulnérabilités** : harmonisation des sévérités, classifications (CWE/OWASP), preuves et localisations issus de sources hétérogènes, afin de produire une vision cohérente du risque.
- **Scoring de risque** : calcul d'un score agrégé par projet et par version, permettant de prioriser les efforts de remédiation selon des critères objectifs.
- **Workflow de gestion des vulnérabilités** : gestion des états (ouvert, confirmé, corrigé, faux positif), assignation à des responsables et suivi par commentaires, pour structurer et tracer la remédiation.
- **Enrichissement par IA locale (LLM)** : contextualisation des alertes, proposition de correctifs et assistance à la priorisation via l'action *Assist with AI*, exécutée sur un modèle local garantissant la confidentialité du code.
- **Génération de rapports et exports** : production de livrables PDF et JSON filtrables par période, projet et niveau de sévérité, à destination des équipes de direction et d'audit.
- **Tableau de bord et visualisation** : consolidation des métriques de sécurité, des tendances et des taux de remédiation, avec agrégations optimisées pour maintenir la fluidité de l'interface.
- **API REST et intégration CI/CD** : déclenchement et suivi des scans depuis les pipelines de livraison, avec contrats d'API stables, gestion des quotas et authentification robuste.
- **Notifications et alerting** : diffusion par email, webhooks ou canal temps réel lors de la détection de vulnérabilités critiques, avec mécanismes anti-saturation configurables.

- **Historisation et traçabilité des scans** : conservation de l'historique par projet et par version pour permettre les comparaisons longitudinales, les audits et le suivi de la maturité sécurité dans le temps.

2.3.2 Besoins non fonctionnels

Un besoin non fonctionnel est une condition qui spécifie les critères permettant de juger le fonctionnement d'un système, plutôt que ses comportements spécifiques. À la différence des besoins fonctionnels qui définissent *ce que* le système fait, les besoins non fonctionnels définissent *comment* il le fait, en termes de performance, de fiabilité, de sécurité et de maintenabilité.

- **Performance et latence** : les opérations de consultation doivent rester fluides ; les traitements intensifs (scans, normalisation, IA) sont systématiquement déportés vers des workers asynchrones afin de ne pas impacter la réactivité de l'API.
- **Scalabilité** : la montée en charge est assurée par une multiplication horizontale des workers d'orchestration, sans dégradation de la qualité de service pour les utilisateurs actifs.
- **Disponibilité et continuité de service** : supervision des conteneurs et politique de redémarrage automatique pour maintenir la disponibilité dans les pipelines CI/CD, même en cas de panne d'un service secondaire.
- **Sécurité et confidentialité** : chiffrement des échanges (TLS), protection des secrets (tokens, clés API), cloisonnement des données par projet et journalisation exhaustive des accès aux ressources sensibles.
- **Maintenabilité** : architecture modulaire, code documenté et couverture de tests suffisante pour faciliter l'évolution des moteurs de scan et des règles métier sans régressions.
- **Extensibilité** : intégration de nouveaux scanners ou formats d'alertes via une architecture par connecteurs et un modèle de données ouvert, sans remaniement du cœur applicatif.
- **Ergonomie et adoption** : parcours utilisateurs cohérents, statuts explicites et interface claire, afin de minimiser la friction opérationnelle et favoriser l'adoption par les équipes.
- **Traçabilité et auditabilité** : journaux complets incluant l'identité de l'acteur, le contexte de l'action et l'horodatage, pour répondre aux exigences de conformité et d'investigations post-incident.
- **Portabilité** : déploiement entièrement conteneurisé (Docker Compose), garantissant la reproductibilité des environnements et la compatibilité avec les principaux gestionnaires de dépôts et systèmes CI/CD du marché.
- **Résilience et fiabilité** : traitement idempotent des tâches et mécanismes de reprise automatique, assurant la continuité fonctionnelle en cas de pannes partielles (indisponibilité d'un scanner ou du module IA).

2.3.3 Diagramme global des cas d'utilisation

Les cas d'utilisation représentent les séquences d'actions réalisées par le système qui produisent un résultat observable pour un acteur donné. Ils permettent de modéliser les attentes des utilisateurs, de délimiter le périmètre fonctionnel du système et d'améliorer la compréhension de ses interactions avec l'environnement extérieur.

Le diagramme 2.1 présente les interactions entre les cinq acteurs identifiés et les principales fonctionnalités d’InvisiThreat. Les relations *include* matérialisent des étapes systématiquement nécessaires — par exemple, l’authentification est incluse dans tout déclenchement de scan — tandis que les relations *extend* représentent des comportements conditionnels, comme l’export PDF qui étend la consultation des résultats selon le rôle et le contexte.

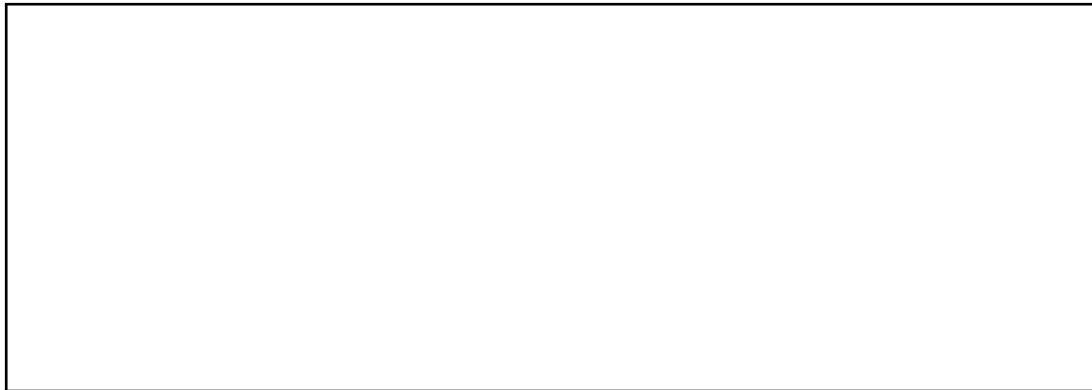


FIGURE 2.1 – Diagramme de cas d’utilisation global d’InvisiThreat.

2.4 Gestion du projet avec Scrum

Le Sprint 0 structure le Product Backlog et prépare la planification des sprints à venir. La priorisation s’appuie sur la criticité sécurité, les dépendances techniques et la valeur métier de chaque fonctionnalité.

2.4.1 Product Backlog

Le Product Backlog formalise la traduction des besoins en unités de valeur livrables. Dans une logique Scrum, il organise le périmètre autour d’epics, de features et de user stories, tout en respectant une priorisation fondée sur le risque et la valeur attendue.

2.4.1.1 Structuration en epics

Les epics regroupent le backlog par grands domaines fonctionnels. L’epic *Authentification et gouvernance* couvre la sécurité des accès et la gestion des rôles. L’epic *Gestion des projets* englobe la configuration des cibles et des dépôts. L’epic *Scans SAST/DAST/SCA* porte sur l’orchestration technique des analyses, tandis que l’epic *Résultats, workflow et IA* regroupe la normalisation, le scoring, la remédiation assistée et les exports. Cette structuration favorise la lisibilité du backlog et facilite la planification incrémentale.

2.4.1.2 Technique de priorisation du Product Backlog

La priorisation du Product Backlog conditionne directement la valeur livrée à chaque sprint. Trois critères complémentaires ont été retenus :

- **Business Value** : priorité aux fonctionnalités apportant le maximum de valeur à l'utilisateur final (détection précoce des vulnérabilités, visibilité du risque par projet).
- **Risk Minimizing** : traitement anticipé des éléments les plus complexes ou les plus incertains sur le plan technique (orchestration asynchrone, intégration du module LLM local, pilotage de ZAP par API).
- **MoSCoW** : classification des items selon quatre niveaux — **M** (*Must have*, obligatoire pour la livraison), **S** (*Should have*, fortement souhaitable), **C** (*Could have*, optionnel à valeur ajoutée), **W** (*Would have*, reporté aux versions futures).

2.4.1.3 Synthèse du Product Backlog

Les tableaux suivants présentent l'ensemble des user stories issues du cahier des charges, organisées par domaine fonctionnel et ordonnées selon leur niveau de priorité MoSCoW.

TABLE 2.2 – Product Backlog – Gouvernance et gestion des accès

ID	Feature	US	User Story	Priorité
Module Gouvernance et Accès				
1	Authentification	1.1	En tant qu'administrateur, je veux m'authentifier afin d'accéder à la plateforme et à l'ensemble des fonctionnalités de gouvernance.	M
1	Authentification	1.2	En tant que développeur, je veux m'authentifier afin d'accéder uniquement à mes projets rattachés.	M
2	2FA TOTP	2.1	En tant qu'administrateur, je veux activer l'authentification à deux facteurs afin de renforcer la protection de mon compte.	M
3	RBAC	3.1	En tant qu'administrateur, je veux définir des rôles et des matrices de permissions afin de cloisonner les accès selon les responsabilités.	M
4	Audit logs	4.1	En tant que Security Manager, je veux consulter le journal des actions sensibles afin d'assurer la traçabilité et la conformité.	M
Module Projets et Intégrations				
5	Gestion des projets	5.1	En tant qu'administrateur, je veux créer un projet et définir ses URLs cibles afin de centraliser la configuration des analyses.	M
5	Gestion des projets	5.2	En tant qu'administrateur, je veux configurer les paramètres de scan par projet afin d'adapter les analyses au contexte applicatif.	S
6	Gestion des membres	6.1	En tant qu'administrateur, je veux inviter des membres et leur attribuer des rôles afin de structurer la collaboration sur la plateforme.	S
7	GitHub OAuth	7.1	En tant que développeur, je veux connecter un dépôt GitHub via OAuth afin d'automatiser la collecte des métadonnées de branches.	S
8	Clés API	8.1	En tant que système CI/CD, je veux m'authentifier via une clé API afin de déclencher des scans sans exposer d'identifiants utilisateur.	S

TABLE 2.3 – Product Backlog – Scans et orchestration

ID	Feature	US	User Story	Priorité
Module Scans SAST / SCA / DAST et Orchestration				
9	SAST Semgrep/Bandit	9.1	En tant que développeur, je veux lancer une analyse statique afin de détecter les patterns de code vulnérables avant tout déploiement.	M
10	Règles internes	10.1	En tant que Security Manager, je veux appliquer des règles regex personnalisées afin d'adapter la détection aux spécificités du code interne.	S
11	SCA OSV	11.1	En tant que développeur, je veux analyser les dépendances via OSV afin d'identifier les composants tiers présentant des vulnérabilités connues.	M
12	CLI scanner	12.1	En tant que système CI/CD, je veux déclencher une analyse via la ligne de commande afin de l'intégrer directement dans la chaîne de livraison.	S
13	Orchestration asynchrone	13.1	En tant que système, je veux que les scans soient mis en file d'attente via Celery/Redis afin de garantir la stabilité sous charge élevée.	M
14	DAST OWASP ZAP	14.1	En tant que développeur, je veux lancer une analyse dynamique via OWASP ZAP afin de détecter les failles d'exposition réelles de l'application.	M
15	Normalisation	15.1	En tant que Security Manager, je veux consulter des alertes normalisées selon une sévérité unifiée afin de prioriser la remédiation efficacement.	M

TABLE 2.4 – Product Backlog – Workflow, IA, reporting et CI/CD

ID	Feature	US	User Story	Priorité
Module Workflow, IA et Reporting				
16	Assistant IA local	16.1	En tant que développeur, je veux obtenir une explication et une recommandation de correction via <i>Assist with AI</i> afin d'accélérer la remédiation.	M
17	Scoring de risque	17.1	En tant que Security Manager, je veux consulter un score de risque par projet afin de prioriser les efforts de sécurité selon des critères objectifs.	M
18	Workflow vulnérabilités	18.1	En tant que développeur, je veux changer l'état d'une vulnérabilité et l'assigner à un responsable afin de structurer le processus de remédiation.	M
19	Dashboard global	19.1	En tant que Security Manager, je veux visualiser un tableau de bord consolidé afin d'avoir une vue d'ensemble du niveau de risque sur l'ensemble des projets.	S
20	Détail des vulnérabilités	20.1	En tant que développeur, je veux consulter les preuves et le contexte d'une alerte afin de comprendre son impact et de la reproduire.	M
21	Génération de rapports	21.1	En tant que Security Manager, je veux générer un rapport PDF ou JSON par projet afin de le partager lors des revues de sécurité.	S
22	Historique comparatif	22.1	En tant que Security Manager, je veux comparer les résultats de scan entre versions afin de mesurer la progression de la maturité sécurité.	S
Module CI/CD et Notifications				
23	Déclenchement CI/CD	23.1	En tant que système CI/CD, je veux déclencher automatiquement un scan sur commit ou merge request afin d'intégrer la sécurité au pipeline de livraison.	M
24	Notifications temps réel	24.1	En tant que développeur, je veux recevoir des notifications instantanées lors de la détection de vulnérabilités critiques afin de réagir sans délai.	S
25	Politiques d'alerte	25.1	En tant que Security Manager, je veux configurer des seuils d'alerte par sévérité afin d'adapter le niveau de notification au contexte du projet.	S
26	Intégration Slack	26.1	En tant que Security Manager, je veux recevoir les alertes critiques dans un canal Slack afin de centraliser les notifications dans l'outil de communication d'équipe.	W

2.4.2 Planification des sprints

La planification de sprint répond à deux questions fondamentales : qu'est-ce qui peut être livré à l'issue de ce sprint ? Et de quelle façon le travail sélectionné sera-t-il conduit ? Pour InvisiThreat, la durée totale du projet a été estimée à **quatre mois**, découpée en cinq sprints de deux à trois semaines chacun, conformément aux préconisations Scrum pour des projets de taille intermédiaire.

La répartition des responsabilités au sein de l'équipe est la suivante :

- **Stagiaire 1** : développement backend, module IA/LLM, DevSecOps et orchestration des scans.
- **Stagiaire 2** : développement frontend, tableaux de bord et intégration CI/CD.
- **Encadrant professionnel** : supervision technique et validation des choix d'architecture.
- **Encadrant académique** : suivi méthodologique et validation scientifique.

Le planning détaillé est présenté au tableau 2.5. La progression suit une logique de construction en couches : fondations techniques, puis gouvernance des accès, puis gestion des projets, puis analyses de sécurité, et enfin restitution enrichie et automatisation.

TABLE 2.5 – Planification et livrables des sprints InvisiThreat

Sprint	Objectif		Contenu	Livrables	Durée
0	Cadrage et fondations	et	Initialisation du dépôt, configuration Docker Compose, modélisation de la base de données, mise en place des squelettes FastAPI et React.	Socle technique opérationnel, schéma PostgreSQL validé, services démarrables.	2 semaines
1	Sécurité et gouvernance	et	Authentification JWT, RBAC, 2FA TOTP, audit logs, politiques de sécurité.	Module d'authentification sécurisé et journalisation opérationnelle.	3 semaines
2	Projets et intégrations	et	CRUD projets et membres, intégration GitHub OAuth, gestion des clés API, notifications de base.	Gestion de projets fonctionnelle, intégration GitHub et clés API opérationnelles.	3 semaines
3	SAST et SCA		Intégration Semgrep et Bandit, règles regex internes, analyse OSV, CLI scanner, normalisation des alertes.	Pipeline SAST/SCA opérationnel, CLI fonctionnel.	3 semaines
4	DAST, orchestration et IA		Intégration OWASP ZAP, workers Celery/Redis, suivi temps réel, scoring de risque, workflow de vulnérabilités, assistant LLM local.	Pipeline DAST asynchrone, scoring de risque, workflow et recommandations IA.	3 semaines

2.5 Étude technique

2.5.1 Environnement matériel

Le projet a été réalisé sur deux postes de travail homogènes, un par stagiaire, afin de garantir des conditions de développement et de test reproductibles. Ce dimensionnement répond à la nature des traitements effectués : l'analyse statique nécessite un accès rapide au système de fichiers et une charge CPU stable, tandis que l'analyse dynamique sollicite davantage la mémoire et la latence réseau pour piloter les scans actifs. L'isolation des conteneurs — notamment OWASP ZAP — dans des processus dédiés limite la contention avec l'API et l'interface, et contribue à stabiliser les temps de réponse observés lors des tests.

TABLE 2.6 – Configuration matérielle des postes de travail

Élément	PC 1 – Wiem Bziouich	PC 2 – Nouha Boughnim
Processeur	Intel Core i7	Intel Core i7
RAM installée	16 Go	16 Go
Système	Windows 11 Pro 64 bits	Windows 11 Pro 64 bits
Stockage	SSD 512 Go	SSD 512 Go

2.5.2 Environnement logiciel

Le tableau 2.7 regroupe l'ensemble des outils utilisés par l'équipe pour le développement, les tests, la modélisation et la gestion du projet. Ces outils constituent l'environnement de travail au sens ingénierie logicielle, distinct de la stack technique déployée en production.

TABLE 2.7 – Environnement logiciel utilisé pour le développement

Logo	Outil	Description et rôle dans le projet
	VS Code	IDE principal pour le développement Python et JavaScript, avec extensions de linting, d'intégration Git et de test des APIs REST.
	Git / GitHub	Contrôle de version distribué, collaboration par branches et revue de code pour assurer la traçabilité des évolutions.
	Docker Desktop	Exécution locale des conteneurs pour reproduire fidèlement l'environnement de production et éviter les conflits de dépendances.
	Postman	Test des endpoints REST, validation des payloads et reproduction des scénarios de scan pour vérifier les contrats d'API.
	Draw.io	Modélisation des diagrammes UML : cas d'utilisation, déploiement et architecture logique.
	Trello / Jira / ClickUp	Gestion du Product Backlog, suivi de l'avancement des sprints et tableaux Kanban partagés entre les membres de l'équipe.
	Chrome / Firefox DevTools	Validation de l'interface React, inspection des requêtes réseau et analyse des performances côté client.
	venv / pip	Isolation des dépendances Python par environnement virtuel, garantissant la cohérence entre les deux postes de développement.
	Node.js / npm	Gestion des dépendances et build du frontend React, avec contrôle précis des versions des bibliothèques côté client.

2.5.3 Technologies et outils utilisés

Le tableau 2.8 présente la stack technique déployée pour assurer la détection, la normalisation et la restitution des résultats de sécurité. Chaque choix est justifié par des exigences de performance, de sécurité et de maintenabilité, et accompagné d'une mise en perspective par rapport aux alternatives.

TABLE 2.8 – Stack technique d’InvisiThreat – technologies et justifications

Logo	Technologie	Rôle, justification et contraintes
 python™	Python FastAPI Backend API	/ Expose une API REST asynchrone capable de traiter des requêtes concurrentes sans blocage lors des scans longs. FastAPI apporte un typage strict et une validation automatique des schémas (Pydantic), ce qui réduit les erreurs d’intégration par rapport à Flask. Sa structure légère est mieux adaptée qu’un framework Django pour un service d’orchestration découplé.
	React Frontend	Bibliothèque de construction d’interfaces riches et réactives, adaptée à la visualisation temps réel des états de scan. Son écosystème étendu et sa flexibilité de composition des composants le distinguent de Vue ou Angular pour des tableaux de bord complexes. La gestion d’état est assurée via hooks et contextes React.
	PostgreSQL Base de données	Assure la persistance transactionnelle (ACID) et les agrégations avancées nécessaires à l’historisation des scans et aux analyses de tendances par sévérité. Offre une richesse fonctionnelle supérieure à MySQL pour les requêtes analytiques. La contrainte principale est l’optimisation des index sur de grands volumes d’alertes.
	Redis Broker & cache	Utilisé comme broker de messages pour Celery et stockage temporaire des états de scan (progression, files d’attente, verrous distribués). Sa nature en mémoire garantit des temps d’accès très faibles et décharge PostgreSQL pour les données éphémères. La stratégie d’expiration des clés constitue la contrainte principale.
	Celery Orchestration async.	Exécute les analyses SAST, SCA et DAST dans des workers isolés sans bloquer l’API principale. Fournit un mécanisme de file d’attente, de retries configurables et de suivi d’exécution adapté aux tâches longues. La supervision des workers et la gestion fine de la concurrence constituent les contraintes opérationnelles majeures.
	Socket.IO Temps réel	Assure la diffusion en temps réel des états de scan et des notifications critiques vers le frontend, éliminant la latence d’une interrogation périodique (polling). La gestion des connexions persistantes et la montée en charge représentent les contraintes principales.
	Docker / Docker Compose Conteneurisation	Constitue le socle de déploiement de l’ensemble des services, garantissant la reproductibilité des environnements et la séparation des composants critiques. Réduit la surcharge par rapport aux machines virtuelles et s’intègre naturellement aux pratiques DevSecOps. La

2.5.4 Architecture du système

2.5.4.1 Architecture logicielle

L'architecture logicielle d'InvisiThreat repose sur une séparation claire des responsabilités en quatre couches fonctionnelles :

- **Couche API** : gestion des projets, des accès utilisateurs, du déclenchement des scans et de l'exposition des résultats via des endpoints REST documentés.
- **Couche d'orchestration** : exécution asynchrone des analyses longues (SAST, SCA, DAST) via des workers Celery, avec file d'attente Redis et suivi de progression.
- **Couche de persistance** : stockage normalisé des vulnérabilités, de l'historique des scans et des artefacts d'audit dans PostgreSQL.
- **Couche de présentation** : interface React pour la visualisation, le workflow de remédiation et les notifications temps réel via Socket.IO.

Le flux principal suit une chaîne continue : déclenchement d'un scan, exécution asynchrone via les workers, normalisation des alertes vers le modèle commun, calcul du scoring de risque, enrichissement par le module IA, puis exposition des résultats via les endpoints REST et le canal temps réel.

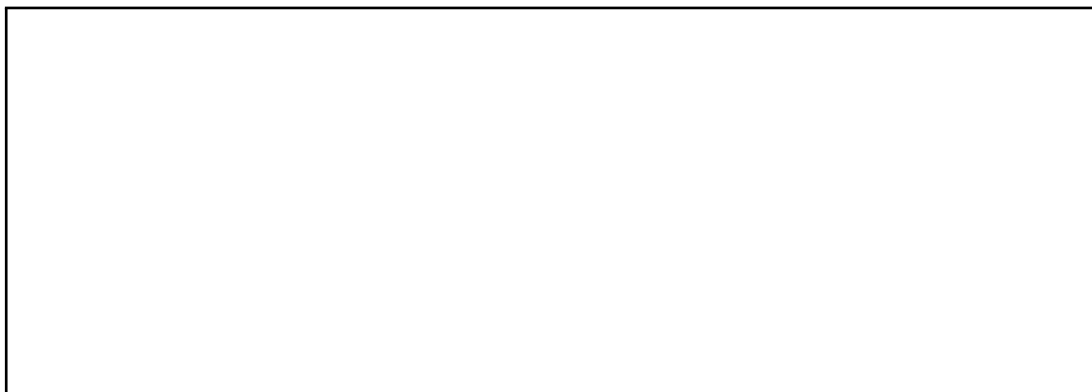


FIGURE 2.2 – Architecture logicielle en couches d'InvisiThreat.

2.5.4.2 Architecture physique

Sur le plan physique, InvisiThreat est déployé via **Docker Compose** selon une organisation en trois niveaux :

- **Niveau client** : navigateur React servi par un conteneur Nginx, communiquant avec le backend via HTTP/REST et Socket.IO.
- **Niveau serveur d'application** : conteneurs FastAPI, workers Celery, broker Redis et module IA Ollama, isolés et communicants par réseau Docker interne.
- **Niveau données** : conteneur PostgreSQL pour la persistance, avec volumes montés pour la durabilité des données entre redémarrages.

InvisiThreat adopte une architecture de **monolithe modulaire** côté backend, enrichie par une orchestration asynchrone. Les moteurs de scan (ZAP, Semgrep, Bandit) et le module IA sont

isolés dans des conteneurs dédiés, sans imposer la complexité opérationnelle d'une architecture microservices complète. La scalabilité horizontale est assurée par la multiplication des workers Celery ; la tolérance aux pannes est renforcée car la défaillance d'un scanner n'interrompt pas les services de consultation ; et l'observabilité (logs structurés, métriques, audit trail) garantit une traçabilité de bout en bout.

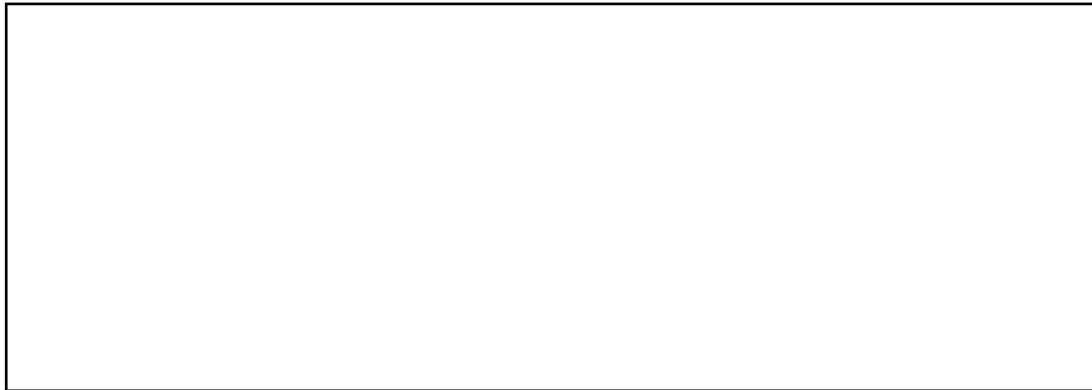


FIGURE 2.3 – Architecture physique – déploiement Docker Compose d'InvisiThreat.

2.6 Diagramme de déploiement

Le diagramme de déploiement illustre l'organisation conteneurisée de la plateforme via Docker Compose, les flux réseau entre services et l'exposition des ports applicatifs (API FastAPI, frontend React, moteur ZAP, module Ollama). Il permet de visualiser l'isolation des composants critiques et les dépendances de communication entre niveaux.



FIGURE 2.4 – Diagramme de déploiement Docker Compose d'InvisiThreat.

2.7 Conclusion

Ce chapitre a posé les fondations méthodologiques et techniques du projet InvisiThreat. L'identification des acteurs et la spécification exhaustive des besoins fonctionnels et non fonctionnels ont permis de délimiter précisément le périmètre de la plateforme. La structuration du Product Backlog selon une priorisation MoSCoW et la planification en cinq sprints garantissent une progression maîtrisée, alignée sur la valeur métier et la gestion du risque

technique. Enfin, l'étude technique a confirmé la cohérence de la stack retenue avec les exigences DevSecOps : architecture modulaire, orchestration asynchrone, enrichissement par IA locale et déploiement intégralement conteneurisé.

Le chapitre suivant présente le **Sprint 1**, consacré à la sécurité et à la gouvernance : authentification JWT, contrôle d'accès par rôles, double facteur TOTP et journalisation des actions sensibles.

Chapitre

3

Sprint 1 – Sécurité et gouvernance

3.1 Introduction

Ce sprint vise à mettre en place le socle de sécurité et de gouvernance de la plateforme InvisiThreat. L'objectif est de garantir un accès sécurisé, une traçabilité complète des actions sensibles et un modèle d'autorisation cohérent pour les différents profils d'utilisateurs.

3.2 Objectifs du sprint et planification des tâches

3.2.1 Objectifs du sprint

- mettre en place l'authentification JWT et la gestion des sessions ;
- définir un modèle RBAC pour les rôles *Admin*, *Security Manager*, *Développeur* et *Viewer* ;
- intégrer la 2FA TOTP pour les comptes sensibles ;
- assurer la traçabilité via des audit logs.

3.2.2 Sprint Backlog

TABLE 3.1 – Sprint Backlog – Sprint 1

ID	User Story	Priorité
S1-01	En tant qu'utilisateur, je peux m'authentifier et obtenir un jeton JWT.	M
S1-02	En tant qu'administrateur, je peux gérer les rôles et permissions (RBAC).	M
S1-03	En tant qu'administrateur, je peux activer la 2FA TOTP pour renforcer la sécurité.	M
S1-04	En tant que Security Manager, je peux consulter l'audit des actions sensibles.	M

3.3 Technologies et concepts utilisés

3.3.1 JWT et OAuth2 (FastAPI Security)

L'authentification repose sur des jetons JWT signés côté serveur, avec un mécanisme de rafraîchissement contrôlé. L'usage d'OAuth2 dans FastAPI standardise les flux d'authentification et assure une intégration propre avec les endpoints sécurisés.

3.3.2 TOTP et RBAC

La 2FA TOTP ajoute un facteur de sécurité basé sur des codes temporaires, tandis que le RBAC définit une matrice de permissions fine par rôle. Cette combinaison garantit que chaque action critique est limitée au bon profil et journalisée.

3.4 Analyse et spécification des besoins

3.4.1 Diagramme de cas d'utilisation du sprint

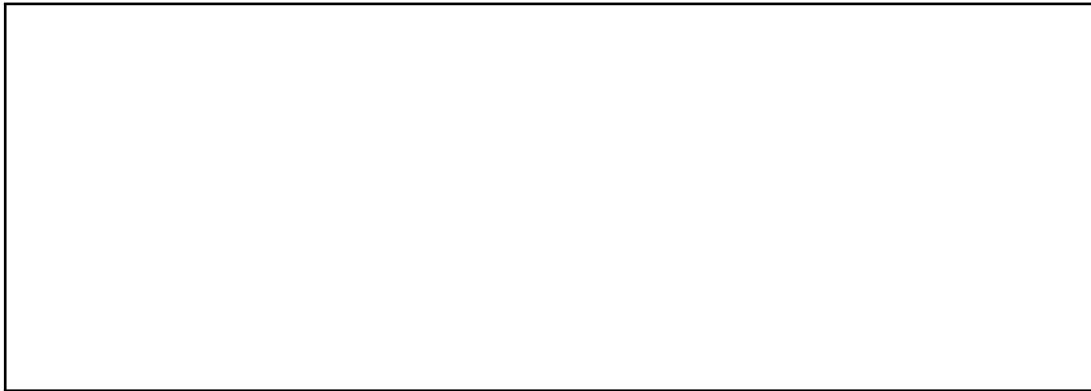


FIGURE 3.1 – Diagramme de cas d'utilisation – Sprint 1 (placeholder).

3.4.2 Description détaillée des cas d'utilisation

Les cas d'utilisation principaux concernent l'authentification (connexion, renouvellement de jeton), la gestion des rôles et permissions, l'activation de la 2FA et la consultation des journaux d'audit. Chaque action critique déclenche une trace horodatée associée à l'identité de l'utilisateur.

3.5 Modélisation conceptuelle

3.5.1 Diagramme de classes



FIGURE 3.2 – Diagramme de classes – Sprint 1 (placeholder).

3.5.2 Diagrammes de séquence



FIGURE 3.3 – Diagrammes de séquence – Sprint 1 (placeholder).

3.6 Réalisation

3.6.1 Authentification JWT

La couche d'authentification fournit un mécanisme stateless avec génération et validation des jetons, ainsi qu'une politique d'expiration cohérente avec l'usage en CI/CD.

3.6.2 RBAC et gestion des rôles

Les rôles sont modélisés dans une matrice de permissions appliquée à chaque requête. Les endpoints sensibles sont protégés par des garde-fous d'autorisation côté API.

3.6.3 Audit logs et 2FA

Chaque action critique (connexion, changement d'état, configuration) génère un audit log. La 2FA TOTP est activable pour les comptes sensibles afin de limiter le risque de compromission.

3.7 Test et validation

3.7.1 Tests réalisés

Les tests incluent des scénarios unitaires de génération de jetons, des tests d'accès par rôle et des tests d'activation/désactivation de la 2FA.

3.7.2 Validation des fonctionnalités

La validation s'appuie sur des scénarios de bout en bout : création d'un utilisateur, attribution d'un rôle, tentative d'accès aux endpoints et vérification des logs.

3.7.3 Gestion des versions / commits

Le développement a été versionné par branches de fonctionnalités, avec des messages de commit structurés afin de retracer l'évolution du module de sécurité.

3.8 Conclusion

Le Sprint 1 a posé le socle de gouvernance et de sécurité de la plateforme. Les modules d'authentification, de contrôle d'accès et d'audit garantissent une base solide pour les sprints fonctionnels suivants.

3.8.1 Diagramme IA/LLM

Ce diagramme détaille le flux *Assist with AI* : sélection d'une vulnérabilité, appel au module Ollama, génération d'explications et restitution dans l'interface.

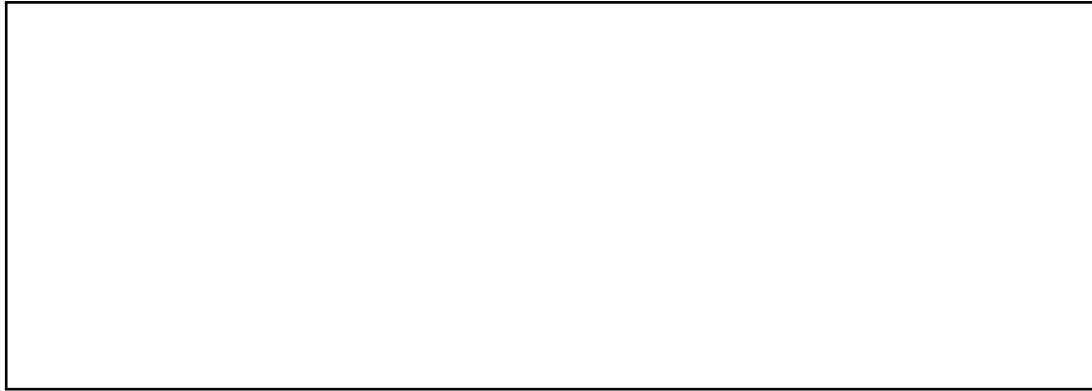


FIGURE 3.4 – Diagramme IA/LLM (placeholder).

3.8.2 Diagramme d'activités

Le diagramme d'activités couvre le cycle complet d'un scan : préparation, exécution asynchrone, normalisation, scoring et notification.



FIGURE 3.5 – Diagramme d'activités (placeholder).

3.9 Configuration du projet

La configuration repose sur des variables d'environnement pour isoler les secrets (clés JWT, identifiants base, URLs de scan) et éviter tout durcissement fragile dans le code source. Cette approche facilite la rotation des secrets et réduit les risques de fuite. Docker Compose joue un rôle central en orchestrant les conteneurs localement et en reproduisant un environnement CI/CD minimal : build, lancement des services et tests automatisés. Des scripts d'automatisation (Makefile ou scripts shell) permettent d'initialiser la base, d'exécuter les migrations et de lancer des scans de démonstration.

3.10 Bonnes pratiques

Le contrôle d'accès **RBAC** applique le principe du moindre privilège et limite la surface d'exposition des fonctions critiques. La gestion des secrets via des variables d'environnement,

avec des mécanismes proches d'un coffre-fort, réduit le risque de compromission. Les scans asynchrones évitent le blocage de l'API et facilitent la mise en file des jobs lourds. Un **audit trail** complet garantit la traçabilité des actions sensibles et répond aux exigences de conformité. Enfin, les tests automatisés intégrés au pipeline DevSecOps sécurisent les évolutions et limitent les régressions fonctionnelles et sécurité.

3.11 Conclusion

Ce chapitre a présenté une étude technique détaillée, justifiant les choix d'outils, d'architecture et de configuration d'InvisiThreat. Les décisions prises assurent la cohérence entre automatisation des scans, sécurité des données et scalabilité. Cette base technique prépare la transition vers la conception détaillée et l'implémentation des modules critiques, en garantissant un alignement fort avec les objectifs DevSecOps.

Chapitre

4

Sprint 2 – Projets et intégrations

4.1 Introduction

Ce sprint est consacré à la gestion des projets et aux intégrations nécessaires pour connecter InvisiThreat à l'écosystème existant. L'objectif est de permettre la création de projets, la configuration des dépôts et l'automatisation des déclenchements via OAuth et clés API.

4.2 Objectifs du sprint et planification des tâches

4.2.1 Objectifs du sprint

- permettre la création et la configuration des projets ;
- intégrer les dépôts GitHub via OAuth ;
- gérer les clés API pour les pipelines CI/CD ;
- initialiser des notifications de base sur les scans.

4.2.2 Sprint Backlog

TABLE 4.1 – Sprint Backlog – Sprint 2

ID	User Story	Priorité
S2-01	En tant qu'administrateur, je peux créer un projet et définir ses URLs cibles.	M
S2-02	En tant que développeur, je peux connecter un dépôt GitHub via OAuth.	S
S2-03	En tant que système CI/CD, je peux déclencher un scan via une clé API.	S
S2-04	En tant que Security Manager, je peux consulter des notifications de scan.	S

4.3 Technologies et concepts utilisés

L'intégration GitHub s'appuie sur le flux OAuth2 afin d'autoriser l'accès aux dépôts sans exposer d'identifiants. Les clés API sont générées et stockées de manière sécurisée pour authentifier les appels automatisés. Le modèle de projet regroupe dépôts, URLs cibles et paramètres de scan afin de centraliser la configuration.

4.4 Analyse détaillée des besoins

4.4.1 Diagramme de cas d'utilisation du sprint



FIGURE 4.1 – Diagramme de cas d'utilisation – Sprint 2 (placeholder).

4.4.2 Description détaillée des cas d'utilisation

Les cas d'utilisation couvrent la création d'un projet, l'association d'un dépôt GitHub, la génération d'une clé API et le paramétrage des URLs de scan. Chaque action est auditée et limitée au périmètre de l'utilisateur.

4.5 Modélisation conceptuelle

4.5.1 Diagramme de classes



FIGURE 4.2 – Diagramme de classes – Sprint 2 (placeholder).

4.5.2 Diagrammes de séquence

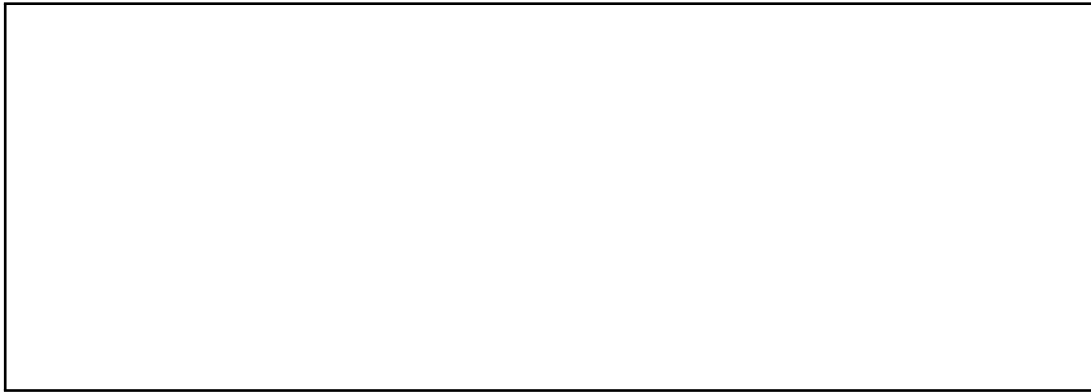


FIGURE 4.3 – Diagrammes de séquence – Sprint 2 (placeholder).

4.6 Réalisation

4.6.1 Gestion des projets et des dépôts

Le module projet centralise les métadonnées (nom, dépôt, URLs, paramètres) et sert de point d'entrée pour les scans. La configuration est persistée afin d'assurer la traçabilité et l'historisation.

4.6.2 Intégrations OAuth et clés API

L'intégration OAuth permet d'accéder aux dépôts GitHub pour récupérer les métadonnées et autoriser les scans. Les clés API offrent un mode d'authentification technique pour les pipelines CI/CD, avec rotation et révocation possibles.

4.7 Test et validation

4.7.1 Tests réalisés

Les tests incluent la création de projets, la connexion OAuth, la génération de clés API et la vérification des droits d'accès selon le rôle.

4.7.2 Validation

La validation s'appuie sur des scénarios CI/CD simulés déclenchant des scans et vérifiant la récupération correcte des projets et dépôts.

4.8 Conclusion

Le Sprint 2 a permis d'ouvrir la plateforme aux intégrations essentielles. La gestion des projets, des dépôts et des clés API prépare le terrain pour les sprints orientés scanners et orchestration.

Chapitre

5

Sprint 3 – SAST et SCA

5.1 Introduction

Ce sprint est dédié à l'intégration des analyses statiques (SAST) et de la composition logicielle (SCA). Il vise à automatiser la détection précoce des vulnérabilités dans le code et les dépendances, tout en normalisant les résultats pour un suivi cohérent.

5.2 Objectifs du sprint et planification des tâches

5.2.1 Objectifs du sprint

- intégrer Semgrep et Bandit pour l'analyse SAST ;
- intégrer OSV pour l'analyse SCA des dépendances ;
- normaliser les résultats dans un modèle unique ;
- exposer un rapport lisible et filtrable.

5.2.2 Sprint Backlog

TABLE 5.1 – Sprint Backlog – Sprint 3

ID	User Story	Priorité
S3-01	En tant que développeur, je peux lancer un scan SAST via Semgrep/Bandit.	M
S3-02	En tant que développeur, je peux analyser les dépendances via OSV.	M
S3-03	En tant que Security Manager, je peux consulter des alertes normalisées.	M
S3-04	En tant que développeur, je peux exporter un rapport de scan.	S

5.3 Analyse détaillée des besoins

5.3.1 Diagramme de cas d'utilisation du sprint



FIGURE 5.1 – Diagramme de cas d'utilisation – Sprint 3 (placeholder).

5.3.2 Description textuelle des cas d'utilisation

Les cas d'utilisation portent sur la soumission d'un dépôt, l'exécution des moteurs SAST/SCA et la consultation des résultats normalisés. L'utilisateur peut filtrer les vulnérabilités par sévérité et accéder aux preuves associées.

5.4 Modélisation conceptuelle

5.4.1 Diagramme de classes



FIGURE 5.2 – Diagramme de classes – Sprint 3 (placeholder).

5.4.2 Diagrammes de séquence

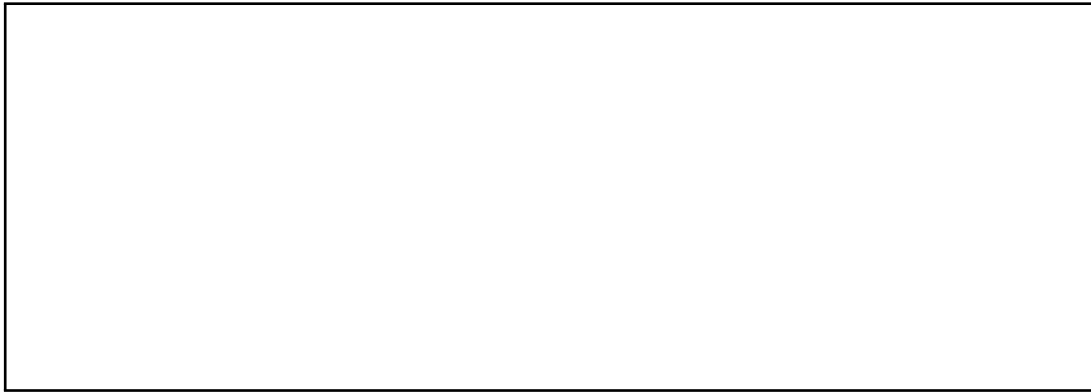


FIGURE 5.3 – Diagrammes de séquence – Sprint 3 (placeholder).

5.5 Réalisation

5.5.1 Intégration SAST (Semgrep, Bandit)

Les moteurs Semgrep et Bandit sont orchestrés via des jobs asynchrones afin d’analyser le code source. Les résultats sont convertis vers un format interne unifié pour faciliter la priorisation.

5.5.2 Intégration SCA (OSV)

L’analyse SCA s’appuie sur OSV pour identifier les dépendances vulnérables et lier chaque alerte à un identifiant CVE/OSV. Les alertes sont regroupées par projet et version.

5.6 Test et validation

5.6.1 Tests réalisés

Les tests incluent l’exécution d’un scan SAST sur un dépôt témoin, l’analyse SCA d’un fichier de dépendances et la vérification du mapping des sévérités.

5.6.2 Validation

La validation repose sur des scénarios où les résultats SAST/SCA sont comparés à des sorties attendues et consolidés dans le tableau de bord.

5.7 Conclusion

Le Sprint 3 a permis d'intégrer la détection SAST/SCA et de normaliser les alertes. Cette base ouvre la voie au Sprint 4, centré sur la détection dynamique et l'orchestration avancée.

Chapitre

6
IA

Sprint 4 – DAST, orchestration et

6.1 Introduction

Ce sprint finalise la chaîne DevSecOps par l'intégration du DAST et l'orchestration asynchrone, tout en consolidant le workflow de vulnérabilités et l'assistance IA. L'objectif est d'apporter une visibilité temps réel et une remédiation priorisée.

6.2 Objectifs du sprint et planification des tâches

6.2.1 Objectifs du sprint

- intégrer OWASP ZAP pour l'analyse dynamique ;
- orchestrer les scans via Celery/Redis avec suivi temps réel ;
- mettre en place le workflow de vulnérabilités et le scoring de risque ;
- activer l'assistance IA pour la remédiation.

6.2.2 Sprint Backlog

TABLE 6.1 – Sprint Backlog – Sprint 4

ID	User Story	Priorité
S4-01	En tant que développeur, je peux lancer un scan DAST via OWASP ZAP.	M
S4-02	En tant que système, je peux orchestrer les scans asynchrones.	M
S4-03	En tant que Security Manager, je peux suivre le workflow des vulnérabilités.	M
S4-04	En tant que développeur, je peux obtenir une recommandation via l'assistant IA.	S

6.3 Analyse détaillée des besoins

6.3.1 Diagramme de cas d'utilisation du sprint



FIGURE 6.1 – Diagramme de cas d'utilisation – Sprint 4 (placeholder).

6.3.2 Description textuelle des cas d'utilisation

Les cas d'utilisation couvrent le déclenchement d'un scan DAST, le suivi d'exécution, la consultation des résultats enrichis et la transition des vulnérabilités dans le workflow. L'assistant IA fournit des explications contextualisées pour accélérer la correction.

6.4 Modélisation conceptuelle

6.4.1 Diagramme de classes



FIGURE 6.2 – Diagramme de classes – Sprint 4 (placeholder).

6.4.2 Diagrammes de séquence

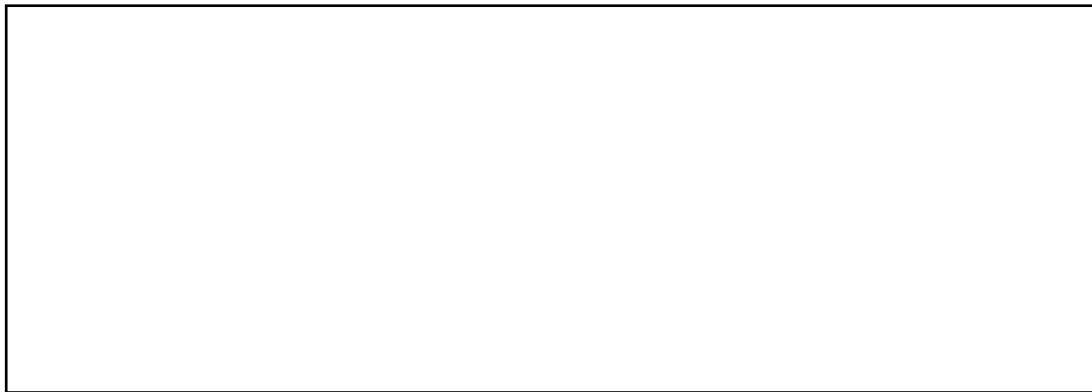


FIGURE 6.3 – Diagrammes de séquence – Sprint 4 (placeholder).

6.5 Réalisation

6.5.1 Intégration DAST (OWASP ZAP)

Le moteur OWASP ZAP est exécuté en mode daemon et piloté via API REST. Les profils de scan sont paramétrables afin d'équilibrer couverture et performance.

6.5.2 Orchestration et workflow

Les scans sont orchestrés via Celery/Redis, avec un suivi de progression et une gestion des files d'attente. Les vulnérabilités sont normalisées, scorées et intégrées dans un workflow d'état (ouvert, confirmé, corrigé, faux positif).

6.6 Test et validation

6.6.1 Tests réalisés

Les tests incluent l'exécution d'un scan DAST sur une cible de test, la vérification des transitions de workflow et l'affichage des notifications en temps réel.

6.6.2 Validation

La validation s'appuie sur des scénarios de bout en bout combinant SAST/SCA/DAST et vérifiant la priorisation, l'historisation et les recommandations IA.

6.7 Conclusion

Le Sprint 4 clôture la chaîne DevSecOps en consolidant l'analyse dynamique, l'orchestration et l'assistance IA. La plateforme dispose désormais d'un pipeline complet et d'un workflow de remédiation exploitable.

6

CONCLUSION GÉNÉRALE

InvisiThreat a permis de formaliser un pipeline de détection combinant SAST, DAST et SCA, de centraliser les résultats et d'offrir un tableau de bord exploitable pour les équipes techniques. L'intégration de FastAPI, PostgreSQL, Redis/Celery et OWASP ZAP a assuré une base solide pour l'automatisation, tandis que React et Socket.IO ont apporté une visualisation réactive de l'état des scans. L'assistant IA local, exposé via *Assist with AI*, renforce la compréhension des vulnérabilités et accélère la remédiation grâce à des recommandations contextualisées.

Les perspectives immédiates incluent l'intégration CI/CD complète (gates de sécurité sur merge), l'amélioration du moteur de corrélation pour réduire les faux positifs et l'extension du périmètre vers l'analyse d'images Docker et d'Infrastructure as Code. L'ajout d'une intégration avec des outils de ticketing (Jira, GitHub Issues) et un pilotage multi-tenant renforcerait la gouvernance et l'adoption à l'échelle.

6

Bibliographie

Dos du rapport (à compléter si nécessaire).